



CONTENTS

1	Cognitive Biases 1	3
1.1	Fundamental Attribution Error	3
1.2	Self-Serving Bias	4
1.3	In-group favoritism	4
1.4	The Bandwagon Effect and Groupthink	5
1.5	The Curse of Knowledge	5
1.6	False Consensus	6
1.7	The Spotlight Effect	7
1.8	The Dunning-Kruger Effect	8
1.9	Confirmation Bias	9
1.10	Survivorship bias	11
2	Longitude and Latitude	13
2.1	Longitude and Latitude	13
2.2	Nautical Mile	15
2.3	Haversine Formula	15
3	Sampling Data	19
3.1	Planning a Study	19
3.1.1	Vocabulary	19
3.1.2	Why Not Always Use a Census?	20
3.2	Planning and Conducting Surveys	21
3.2.1	Biased Sampling	21
3.3	Simple Random Sampling	22

3.3.1	Other Methods of Random Sampling	24
3.4	Bias in Surveys	29
4	Data tables in SQL	31
4.1	Using SQL from Python	34
5	Advanced SQL and Database Management	37
5.1	Keys	37
5.2	Normalization	38
5.2.1	Functional Dependencies	39
5.2.2	1NF	39
5.2.3	2NF	40
5.2.4	3NF	42
5.3	Entity-Relationship Diagram	43
5.4	Relational Algebra Basics	43
5.5	Relating Two Tables	43
6	Diodes, Rectifiers, and Photovoltaics	45
6.1	Silicon	45
6.2	Diodes	47
6.2.1	Rectifiers	50
6.2.2	Photovoltaics	50
A	Answers to Exercises	53
Index		57

Cognitive Biases 1

In this section, we are going to take a look at research findings about *cognitive biases*. These are universal quirks found in the human thought process. Cognitive biases are a bit different from other kinds of biases, such as racial biases. Everyone, regardless of nationality, race, or gender is subject to these cognitive traps. You might be wondering, why do I need to learn about cognitive science in order to be an engineer? The most important tool we have as problem solvers is our own minds. We are going to be looking at ways that our minds can trip us up.

Our brains were designed over millions of years by the evolutionary process. The resulting mind is an amazing and powerful tool, but it is not flawless. The human brain has tendencies (or biases) that nudge us toward bad judgment and poor decisions.

When someone first gave you a hammer they handed it over with a warning: "Don't hit your thumb!" No matter how careful you are with the hammer, at some point you will still hit your thumb. It's the same with cognitive biases. In the course of life, all of us will fall prey to these cognitive biases. Knowing about them is the first step in protecting ourselves.

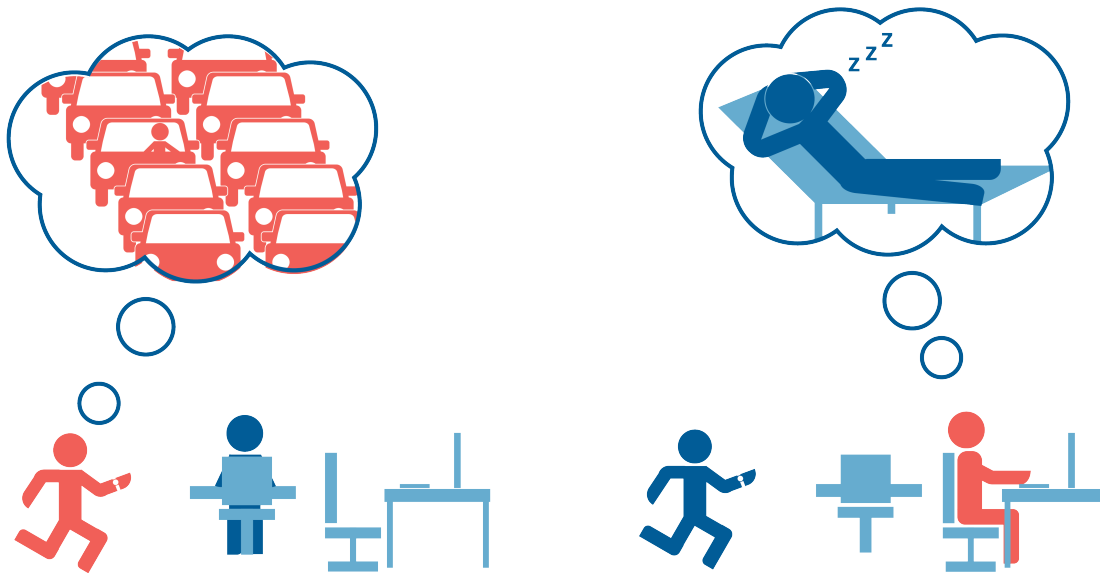
It would be irresponsible to teach you powerful ideas without also teaching you about the cognitive biases that follow them. There are about 50 that you should know about, but let's start with only a few.

1.1 Fundamental Attribution Error

You tend to attribute the mistakes of another person to their character, but attribute your own mistakes to the situation.

If someone asks you "Why were you late for work today?" you are likely to have an excuse, such as "I got stuck in a crazy traffic jam."

However, if you notice your coworker is late for work, you are likely to think "My coworker is lazy."



The solution? Cut people some slack. You probably don't know the whole story, so assume that their character is as strong as yours.

Or maybe you also need to hold yourself to a higher standard? Do you find yourself frequently rationalizing your bad judgment, lateness, or rudeness? This could be an opportunity for you to become a better person whose character is stronger regardless of the situation.

1.2 Self-Serving Bias

Self-serving bias is when you blame the situation for your failures, but attribute your successes to your strengths.

For example, when asked “Why did you lose the match?” you are likely to answer “The referee wasn’t fair.” When you are asked “Why did you win the match?” you are likely to answer “Because I have been training for weeks, and I was very focused.”

This bias tends to make us feel better about ourselves, but it makes it difficult for us to be objective about our strengths and weaknesses.

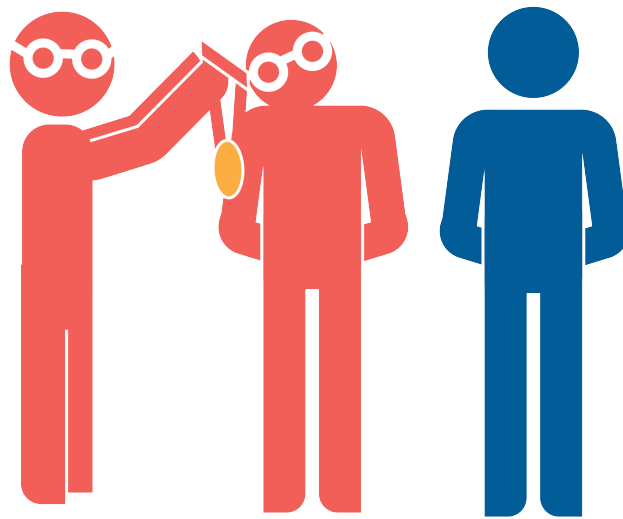
1.3 In-group favoritism

In-group favoritism: We tend to favor people who are in a group with us over people who are not in groups with us.

When asked “Who is the better goalie, Ted or John?”

If Ted is a Star Trek fan like you, you are likely to think he is also a good goalie.

As you might imagine, this unconscious tendency is the source of a lot of subtle discrimination based on race, gender, age, and religion. As we mentioned earlier, racial bias isn’t a cognitive bias, but one can still feed into the other.



1.4 The Bandwagon Effect and Groupthink

The bandwagon effect is our tendency to believe the same things that the people around us believe. This is how fads spread so quickly: one person buys in, and then the people they know have a strong tendency to buy in as well.

Groupthink is similar: To create harmony with the people around us, we go along with things we disagree with.

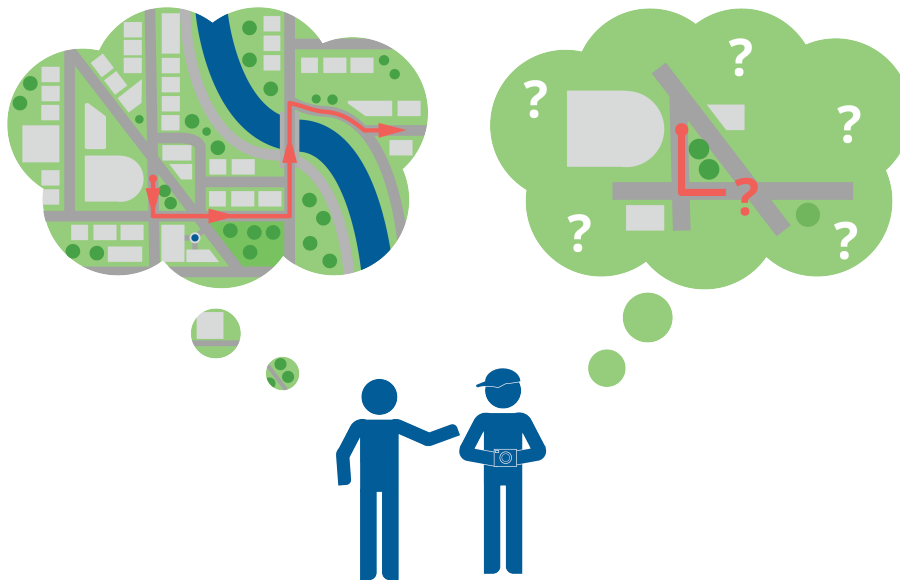
It takes a lot of perspective to recognize when those around us are wrong. And it takes even more courage to openly disagree with them.

1.5 The Curse of Knowledge

Once you know something, you tend to assume everyone else knows it too.

This is why teaching is sometimes difficult; a teacher will assume that everyone in the audience already knows the same things the teacher knows.

For example, imagine a local who has lived in a city for years giving directions to a tourist. The local has an in-depth understanding of the city, and gives overly quick and detailed instructions. The tourist politely smiles and nods, but stopped following after the local began listing unfamiliar street names and landmarks.



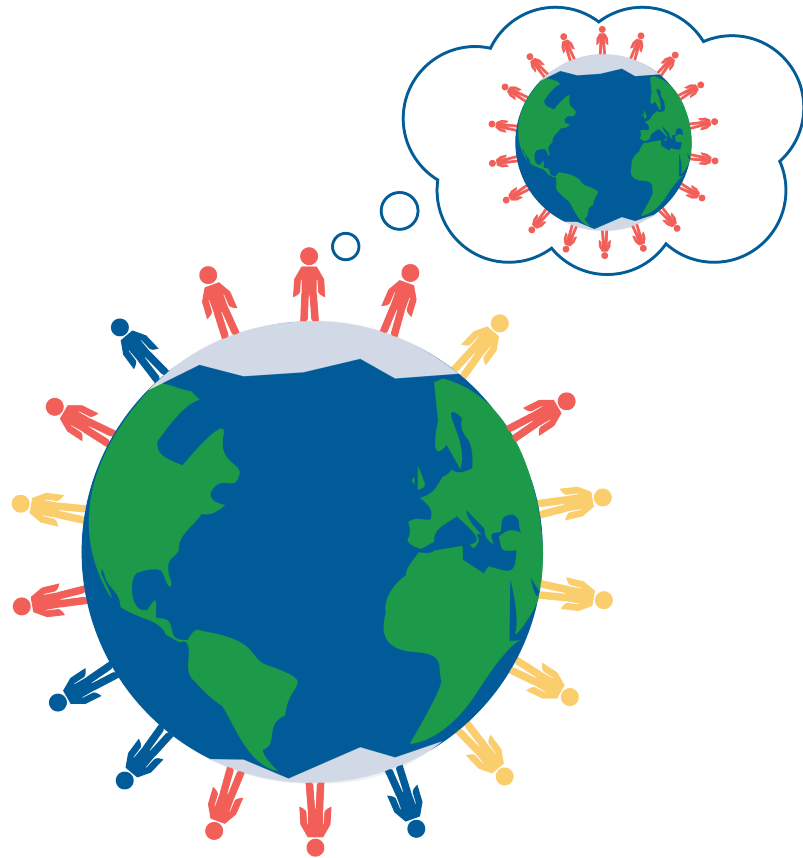
When we learn that a friend doesn't know something that we know, we are often very surprised. This surprise can sometimes manifest as hurtful behavior.

When we find a gap in our friend's knowledge, we try to remind ourselves that the friend certainly knows many things that we don't. We also try to imagine how it would feel if they teased us for our ignorance.

1.6 False Consensus

We tend to believe that more people agree with us than is actually the case. For example, if you are a member of a particular religion, you tend to overestimate the percentage of people in the world who are members of that religion.

When people vote in elections, they are often surprised when their preferred candidate loses. "Everyone, and I mean EVERYONE, voted for Smith!" they yell. "There must have been a mistake in counting the votes."

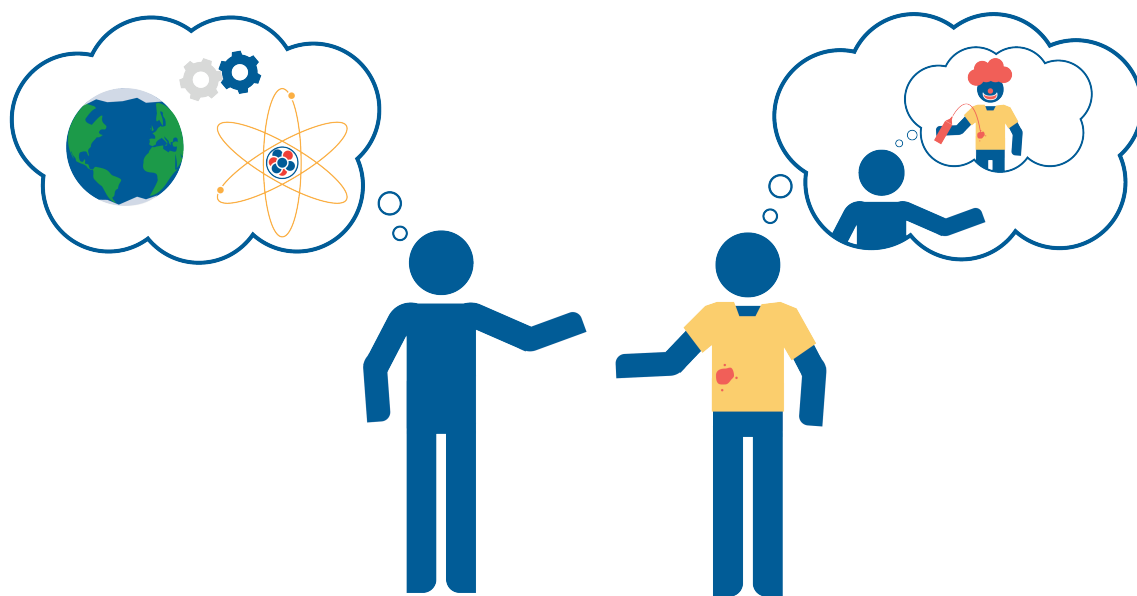


1.7 The Spotlight Effect

You tend to overestimate how much other people are paying attention to your behavior and appearance.

Think of six people that you talked to today. Can you even remember what shoes most of them were wearing? Do you care? Do you think any of them remember which shoes you wore today?

There is an old saying, “You would worry a lot less about how people think of you, if you realized how rarely they do.”



1.8 The Dunning-Kruger Effect

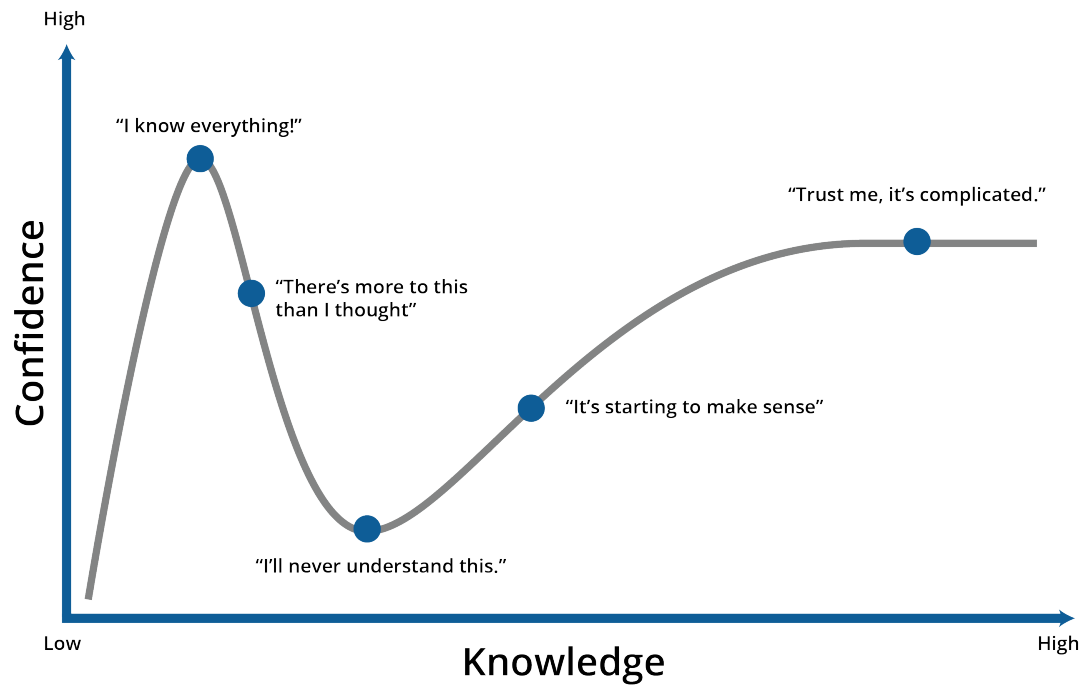
The less you know, the more confident you are.

When a person doesn't know all the nuance and context in which a question is asked, the question seems simple. Thus, the person tends to be confident in their answer. As they learn more about the complexity of the space in which the question lives, they often realize the answer is not nearly so obvious.

For example, a lot of people will confidently proclaim "Taxes are too high! We need to lower taxes." An economist who has studied government budgets, deficits, history, and monetary policy, might say something like "Maybe taxes *are* too high. Or maybe they are too low. Or maybe we are taxing the wrong things. It is a complex question."

When we are talking with people about a particular topic, we do our best to defer to the person in the conversation who we think has the most knowledge in the area. If we disagree with the person, we try to figure out why our opinions are different.

Similarly, you should assume that any opinion that is voiced in an internet discussion is, at best, wildly over-simplified. If you really care about the subject, read a book by a respected expert. Yes, a whole book — there are few interesting topics that can be legitimately explained in under 100 pages.

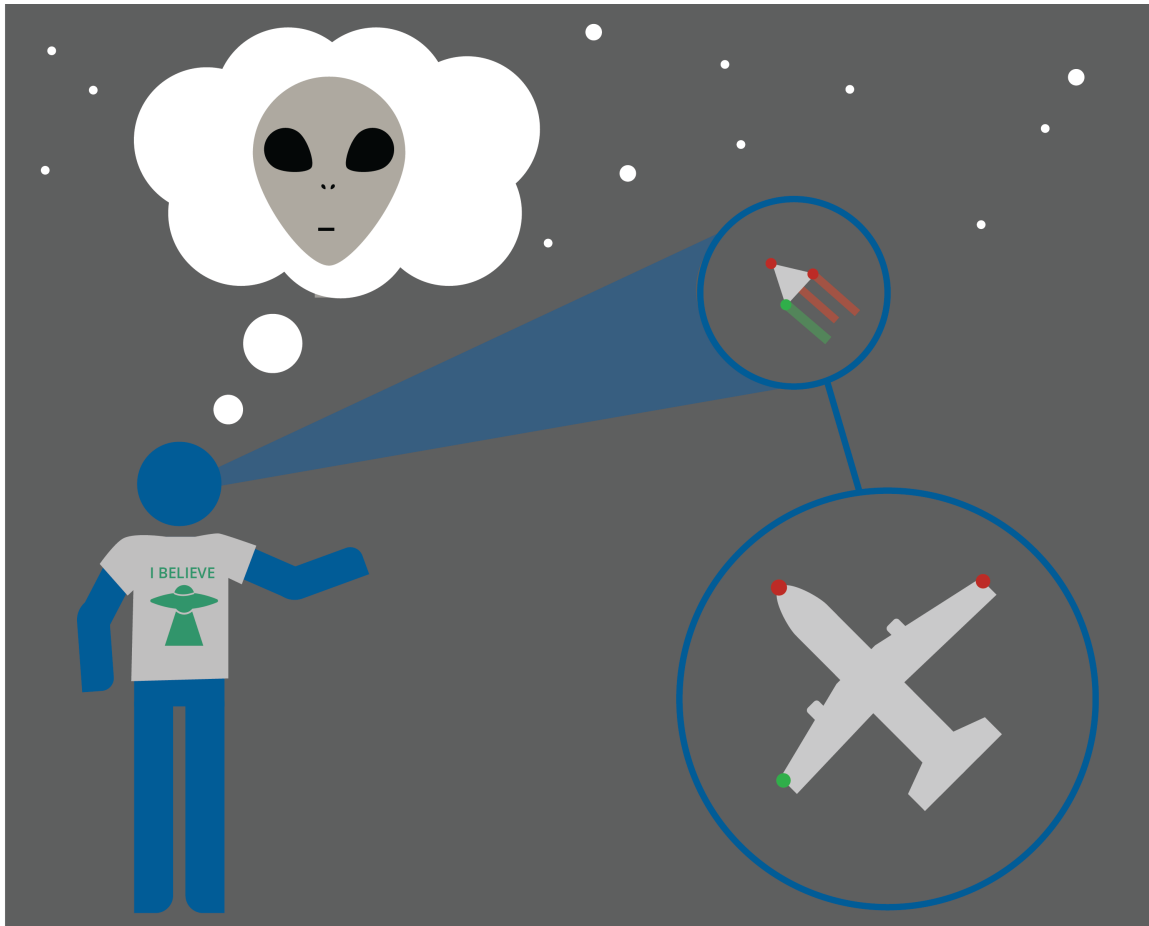


1.9 Confirmation Bias

You tend to find and remember information that supports beliefs you already have. You also tend to avoid and dismiss information that contradicts your beliefs.

If you believe that intelligent creatures have visited from other planets, you will tend to look for data to support your beliefs. When you find data that shows that it is just too far for any creature to travel, you will try to find a reason why the data is incorrect.

Confirmation bias is one reason why people don't change their beliefs more often.



Confirmation bias wrecks many, many studies. The person doing the study often has a hypothesis that they believe and very much want to prove true. It is very tempting to discard data that doesn't support the hypothesis. A person may even throw all the data away and experiment again and again until they get the result they want.

When you design an experiment, you must describe it explicitly before you start. You must tell someone: "If the hypothesis I love is incorrect, the results will look like this. If the hypothesis I love is correct, the results will look like that. And if the results look any other way, I have neither proved nor disproved the hypothesis."

Once the experiment is underway, you must not change the plan and you must not discard any data.

This is scientific integrity. You should demand it from yourself, and you should expect it from others.

Watch a TED Talk and Learn More About Confirmation Bias: What shapes our perceptions (and misperceptions) about science? In an eye-opening talk, meteorologist J. Marshall Shepherd explains how confirmation bias, the Dunning-Kruger effect and cog-

nitive dissonance impact what we think we know – and shares ideas for how we can replace them with something much more powerful: knowledge.

https://www.ted.com/talks/j_marshall_shepherd_3_kinds_of_bias_that_shape_your_worldview



1.10 Survivorship bias

You will pay more attention to those that survived a process than those who failed.

After looking at several old houses, you might say “In the 1880s, they built great houses.” However, you haven’t seen the houses that were built in the 1880s and didn’t survive. Which houses tended to survive for a long time? Only the great houses – you are basing your opinion on a very skewed sample.



Longitude and Latitude

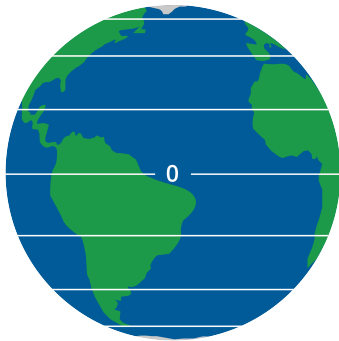
2.1 Longitude and Latitude

The Earth can be represented as a sphere, and the position of a point on its surface can be described using two coordinates: latitude and longitude.

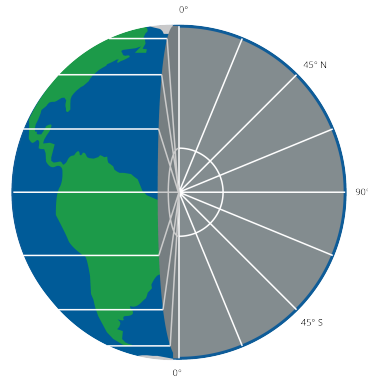


Figure 2.1: A diagram of latitude and longitude.

Latitude is a measure of a point's distance north or south of the equator, expressed in degrees. It ranges from -90° at the South Pole to $+90^\circ$ at the North Pole, with 0° representing the Equator. (See figures [2.2a](#) and [2.2b](#))



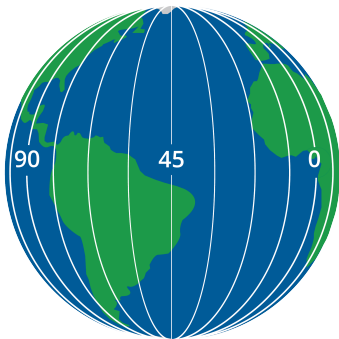
(a) Latitude drawn on the earth.



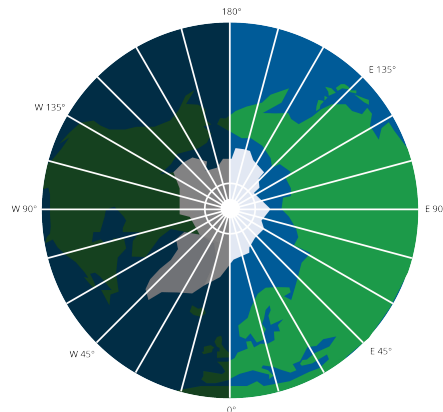
(b) A cross-section of the earth showing latitude.

Figure 2.2: Latitude illustrations

Longitude, on the other hand, measures a point's distance east or west of the Prime Meridian (which passes through Greenwich, England). It ranges from -180° to $+180^\circ$, with the Prime Meridian represented as 0° . (See figures 2.3a and 2.3b)



(a) Longitudinal lines drawn on the earth.



(b) A cross-section of the earth showing longitude.

Figure 2.3: Longitude illustrations.

The coordinates follow a system of latitude and then longitude (in the majority of contexts). Here is the Longitude and Latitude of four different large cities:

- **London, England:** 51° N 0° W
- **New York City, New York, USA:** 40° N 74° W

- **Sydney, New South Wales, Australia:** 33° S 150° E
- **Rio de Janeiro, Brazil:** 22° S 43° W

These coordinates may be broken down further into minute and second components as well, for further geometric accuracy. Each degree ($^{\circ}$) is divided into 60 minutes ($'$). Each minute ($'$) is divided into 60 seconds ($''$). Just like in time. Note some do not

- **London, England:** $51^{\circ} 30' 26''$ N $0^{\circ} 7' 39''$ W
- **New York City, New York, USA:** $40^{\circ} 42' 46''$ N $74^{\circ} 0' 22''$ W
- **Sydney, New South Wales, Australia:** $33^{\circ} 52' 0''$ S $150^{\circ} 12' 0''$ E
- **Rio de Janeiro, Brazil:** $22^{\circ} 54' 0''$ S $43^{\circ} 12' 0''$ W

2.2 Nautical Mile

A nautical mile is a unit of measurement used primarily in aviation and maritime contexts. It is based on the circumference of the Earth, and is defined as one minute ($1/60^{\circ}$) of latitude; in other words, **one minute of latitude is equal to 1 nautical mile**. This makes it directly related to the Earth's geometry, unlike a kilometer or a mile, which are arbitrary in nature. The exact value of a nautical mile can vary slightly depending on which type of latitude you use (e.g., geodetic, geocentric, etc.), but for practical purposes, it is often approximated as 1.852 kilometers or 1.15078 statute miles.

2.3 Haversine Formula

The *haversine formula* is an important equation in navigation for giving great-circle distances between two points on a sphere from their longitudes and latitudes. It is especially useful when it comes to calculating distances between points on the surface of the Earth, which we represent as a sphere for simplicity. See Figure 2.4

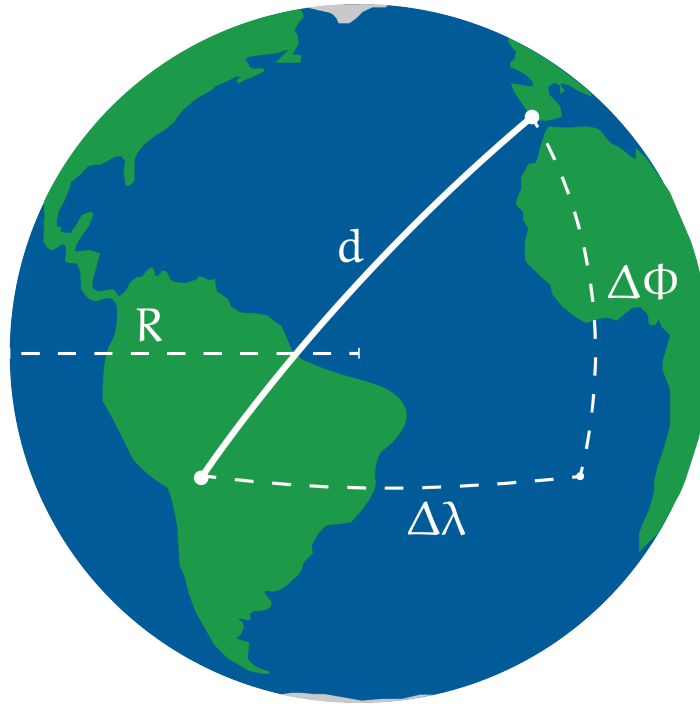


Figure 2.4: A diagram of the haversine formula.

In its basic form, the haversine formula is as follows:

The angle found using haversine $\text{haversine}(\theta) = \sin^2\left(\frac{\theta}{2}\right)$:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

Compute c which determines the angular distance using `atan2`, a computer science tangential function with 2 arguments equivalent to $2 \arcsin(\sqrt{a})$ ¹:

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

Find d , the true distance along the sphere:

¹Note that this function is not available on calculators, but commonly used in computer science programs. You may use the singular argument equivalent in calculations.

$$d = R \cdot c$$

In one line, this is:

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

Here, ϕ_1 and ϕ_2 represent the latitudes of the two points (in radians), $\Delta\phi$ and $\Delta\lambda$ represent the differences in latitude and longitude (also in radians), and R is the radius of the Earth (using whatever unit of distance you desire). The result, d , is the distance between the two points along the surface of the sphere. Notice that d is equal to the formula for the *arc length* of a sector. A sphere is just a bunch of circles with radius r , so it follows that we can use formulas for circles in our spherical calculations.

This is why planes fly arc-shaped paths rather than straight paths from points $A \rightarrow B$. Planes follow the shortest path between two points on a sphere, called a *great-circle route*. On a map, like plane flight-progress maps or the Mercator projection, the straight path looks very arc shaped, but this is just a geographical illusion. This straight distance can be calculated using the haversine formula, ultimately minimizing distance and fuel using the ‘as-the-crow-flies’ path.

Sampling Data

3.1 Planning a Study

Every day, decisions get made based on data. A city council votes on a new transit route based on ridership surveys. A school principal adjusts the lunch schedule based on student feedback. A pharmaceutical company decides whether to release a new drug based on clinical trial results. In every case, the quality of the decision depends entirely on the quality of the data behind it.

To collect data well, you need a good plan! That plan must specify what question you are trying to answer, how you will gather information, and who you will gather this information from.

3.1.1 Vocabulary

Five terms form the foundation of everything that follows. Learn to distinguish them from each other, because you will probably be tested on this.

population The entire group of individuals or items that you want to learn about. The population is defined by the question you are asking.

frame Also called a sampling frame. A list of all the members of the population from which a sample will be drawn. The frame is usually close to the population, but the two are not always identical. A school's enrollment database, a city's voter registration list, and a company's customer records are all examples of frames.

sample The part of the population that is actually studied. You collect data from the sample and use it to draw conclusions about the population.

sample survey The process of collecting information from a sample. The information gathered is then used to make inferences about population parameters.

census The process of collecting information from every member of the population rather than just a sample. A census gives complete information, but it comes at a cost.

The relationship between these terms is important. The population is the full group you care about. The frame is your working list of that group. The sample is the slice of the frame you actually study. A census is what happens when the sample and the population are the same thing or equal in number.

Exercise 1 **Identifying the Population and Sample**

A city's parks department wants to understand how often residents use public parks. Staff randomly select 200 households from the city's water billing records and conduct phone interviews.

Working Space

1. What is the population?
2. What is the frame?
3. What is the sample?
4. Is this a census? Explain.

Answer on Page 53

3.1.2 Why Not Always Use a Census?

A census tells you about everyone in the population. That sounds great, but it is not always realistic. If the group is small, like a football team, you can measure everyone.

Most of the time, a census is too difficult because:

- The population is too large. A company that wants to know what all households watch cannot contact every single one.
- The measurement destroys the item. If testing a battery kills it, checking every battery would waste them all.
- The information gets old too fast. If a company waits months to ask everyone, the results may no longer match what people think.

Exercise 2 Census or Sample?

For each situation, decide whether a census is feasible or whether a sample should be used. Briefly explain your reasoning.

Working Space

Situation	Census or Sample? Explain
A school librarian wants to know which of her 8 student volunteers prefers morning or afternoon shifts.	
A battery manufacturer wants to test the lifespan of batteries by running them until they die.	
A government agency wants to estimate the average commute time of workers across a country of 40 million employed adults.	
A restaurant owner wants to know the dietary restrictions of the 22 guests attending a private event.	

Answer on Page 53

In most studies, sampling is the only practical option, and the quality of the conclusions you draw depends entirely on how well the sample was chosen.

3.2 Planning and Conducting Surveys

Not all samples are fair. How you pick people can make the group represent the population well, or it can leave out important parts.

3.2.1 Biased Sampling

A sample is *biased* if it favors some people or answers over others. Bias does not go away just because the sample is bigger.

These three common methods usually make biased samples.

Judgmental sampling is when one person decides who is included based on their own idea of what looks representative. A school inspector who visits only classrooms she already thinks are good is using judgmental sampling. The choice is based on their opinion.

A *convenience sample* uses whoever or whatever is easiest to reach. A journalist who interviews only people at the bus stop outside his office is using a convenience sample. Those people might not represent all commuters.

A *voluntary response sample* happens when people choose whether to answer. An online poll on a news site will attract mostly visitors with strong opinions. People who do not care much usually do not respond.

All three skip chance. Sampling methods based on random selection give everyone a fair shot and usually make more trustworthy results.

3.3 Simple Random Sampling

A *simple random sample* (SRS) is a sample chosen so every group of the same size has the same chance of being picked.

There are two ways to do it.

With *sampling with replacement*, you put someone back in the pool after choosing them. They could be picked again.

With *sampling without replacement*, you do not put them back. Each person can only be chosen once.

To get an SRS, you need a real random method.

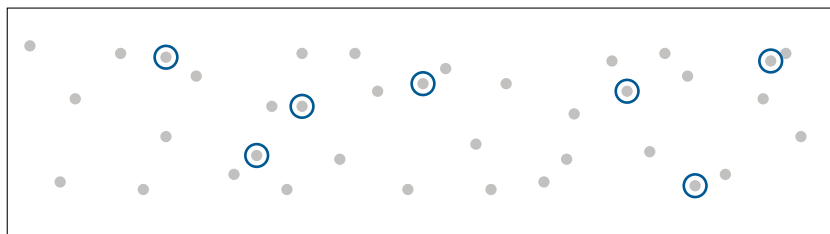


Figure 3.1: In simple random sampling, every individual in the population has an equal chance of being selected, and there is no grouping or pattern to who is chosen.

- Assign every member of the population a number. Use a random number table, entering at any random point and reading numbers sequentially. Select members whose numbers appear, skipping values outside your range and any repeats, until you have the sample size you need.

- Use a calculator. On a TI-84, `MATH` → `PRB` → `5:randInt(` generates random integers. The command `randInt(1,500,30)` produces 30 random integers between 1 and 500. Ignore repeats.
- Use a physical randomization device. Write each member's identifier on an identical slip of paper, mix thoroughly in a container, and draw without looking until the sample is complete.

The essential requirement in every case is that chance dictates who is selected, rather than convenience or judgment. This is what makes the sample fair and the results trustworthy.

Exercise 3 Selecting an SRS

A regional transit authority wants to survey 25 of its 300 bus drivers about workplace safety conditions.

Working Space

1. Describe a complete procedure for selecting an SRS of 25 drivers using a random number table.
2. A supervisor suggests selecting the 25 drivers who are easiest to reach during the next shift change. What type of sample is this, and what bias might it introduce?
3. A colleague suggests selecting 5 drivers at random from each of the authority's 5 depots. Is this an SRS? Explain.

Answer on Page 53

Sampling Real Data from the Web

In this exercise you will load real world population data published by the World Bank, treat all 265 countries and territories as your population, draw a simple random sample, and compare your sample mean to the true population mean.

1. Import the necessary libraries.

```
1 import pandas as pd
```

2. Load the dataset directly from the web and filter to the year 2020.

```
1 url =  
  ↳ "https://raw.githubusercontent.com/datasets/population/main/data/population.csv"  
2 df = pd.read_csv(url)  
3  
4 pop2020 = df[df["Year"] == 2020][["Country Name",  
  ↳ "Value"]].reset_index(drop=True)  
5 print(pop2020.head())  
6 print(f"Total entries: {len(pop2020)}")
```

3. Calculate the true population mean. This is the parameter you are trying to estimate.

```
1 true_mean = pop2020["Value"].mean()  
2 print(f"True population mean: {true_mean:,.0f}")
```

4. Draw an SRS of 20 countries and compute the sample mean.

```
1 sample = pop2020.sample(n=20)  
2 sample_mean = sample["Value"].mean()  
3 print(f"Sample mean (n=20): {sample_mean:,.0f}")
```

5. Run step (d) five times and record the sample mean each time. How close is each to the true mean? Are they all the same?
6. Change the sample size to 80 and repeat step (e). How do the results compare? What does this tell you about the relationship between sample size and sampling error?

3.3.1 Other Methods of Random Sampling

Simple random sampling is not always the most practical or most efficient approach. Three other probabilistic methods appear regularly on the AP exam.

In *systematic sampling*, you list the population, choose a random start, and then pick every k th person. For example, to sample 40 students from 800, start at a random number between 1 and 20, then take every 20th student. This is simple when you have a full list.

In *stratified random sampling*, the population is split into groups called strata. Each group should be similar inside, like students in the same grade or patients in the same ward. Then you take a random sample from each group. This makes sure every part of the

population is represented. This method of random sampling is especially useful in medical studies, market research, political polling, educational testing, etc, where you want to make sure you have enough people from each group to compare them.

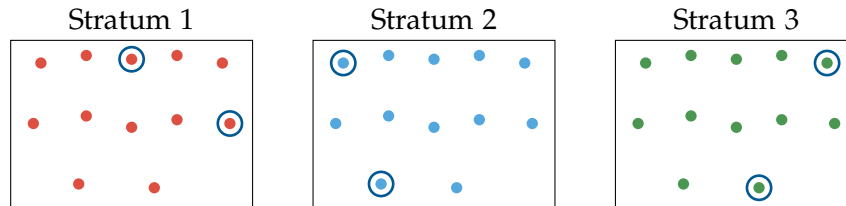


Figure 3.2: Each stratum is a single color, since members within a stratum share a characteristic. A small random sample (circled) is drawn from each stratum.

In *cluster sampling*, the population is split into groups called clusters. Each cluster should be like a mini-version of the whole population. Then you randomly pick some clusters and study everyone inside them. This works well when people are spread out over a wide area and it may be difficult or expensive to reach everyone. For example, a company that wants to survey its customers across the country might randomly select 10 cities and then survey all customers in those cities.

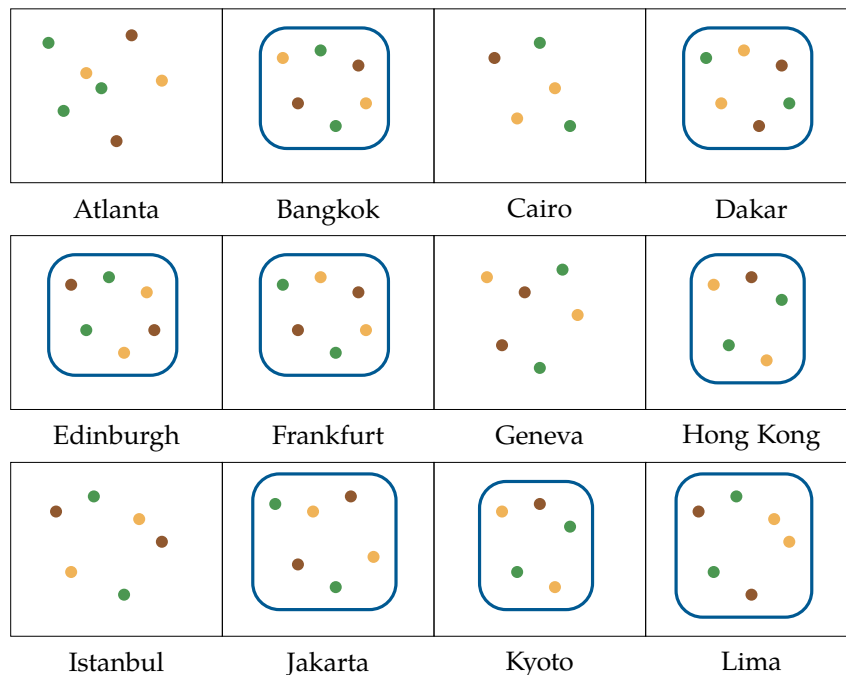


Figure 3.3: Each dot represents a customer and each color represents a different customer segment. Atlanta, Cairo, Geneva, and Istanbul were not selected. The other 8 cities were randomly selected as clusters, and every customer within each selected city becomes part of the sample.

It is important to understand the difference between strata and clusters. *Strata* are groups that are different from each other but similar within, and you sample from all of them. *Clusters* are groups that are similar to the whole population, and you sample only some of them but take everyone inside those.

In *quota sampling*, a non-probability sampling method where researchers select participants until predefined quotas for certain groups (ie: age, gender, ethnicity) are filled. The sample is structured to reflect specific population characteristics, but participants are not chosen randomly. For example, if a population is 60% female and 40% male, a researcher surveys people until they reach those proportions.

In *convenience sampling*, a non-probability sampling method where participants are selected because they are easy to access or available. For example, surveying students who happen to be in a campus cafeteria as you are eating lunch.

Bonus method: *Snowball sampling* is a non-probabilistic method used when the population of interest is difficult or impossible to reach through conventional sampling. The researcher identifies a small number of initial participants who meet the study criteria, and then asks those participants to refer others they know who also qualify. Each wave of referrals generates the next, like a snowball rolling downhill and growing as it goes. It is not a commonly tested method on the AP exam, but it is worth knowing as an example of a non-probabilistic sampling technique that can be useful in certain contexts, such as studying hidden or hard-to-reach populations (e.g., people experiencing homelessness, members of stigmatized groups, or individuals with rare diseases).

Exercise 4 Identifying Sampling Methods

Identify the sampling method used in each scenario: SRS, systematic, stratified, or cluster.

Working Space

Scenario	Method
A supermarket chain selects 10 of its 80 stores at random and surveys every employee in those stores.	
A university randomly selects 50 students from each of its four faculties to participate in a wellbeing survey.	
A water utility selects a random starting meter on its numbered list and then tests every 25th meter for contamination.	
A film festival puts all 900 registered attendee names in a database and uses software to randomly select 90.	

Answer on Page 54

Comparing Sampling Methods in Python

In this exercise you will apply three sampling methods to the same dataset and compare what each one produces. The dataset contains 30 students across three grade levels with their test scores.

Setup: Create the dataset.

```

1 import pandas as pd
2 import numpy as np
3
4 np.random.seed(42)
5
6 data = {
7     "student_id": range(1, 31),
8     "grade": ["9th"]*10 + ["10th"]*10 + ["11th"]*10,
9     "score": (
10         list(np.random.randint(60, 80, 10)) +
11         list(np.random.randint(70, 90, 10)) +
12         list(np.random.randint(80, 100, 10))
13     )

```

```

14 }
15
16 df = pd.DataFrame(data)
17 print(df)

```

Compute the true population mean.

```

1 true_mean = df["score"].mean()
2 print(f"True mean score: {true_mean:.1f}")

```

1. **Simple Random Sample.** Draw an SRS of 9 students.

```

1 srs = df.sample(n=9)
2 print(f"SRS mean: {srs['score'].mean():.1f}")
3 print(srs[["student_id", "grade", "score"]])

```

Run this three times. What grades tend to appear? Does the SRS always represent all three grades?

2. **Stratified Random Sample.** Draw 3 students from each grade level.

```

1 stratified = df.groupby("grade", group_keys=False).apply(
2     lambda x: x.sample(n=3)
3 )
4 print(f"Stratified mean: {stratified['score'].mean():.1f}")
5 print(stratified[["student_id", "grade", "score"]])

```

Run this three times. How does the sample mean compare to the true mean? Is it more or less stable than the SRS?

3. **Cluster Sample.** Randomly select one entire grade level as the cluster.

```

1 chosen_grade = pd.Series(df["grade"].unique()).sample(n=1).values[0]
2 cluster = df[df["grade"] == chosen_grade]
3 print(f"Cluster selected: {chosen_grade}")
4 print(f"Cluster mean: {cluster['score'].mean():.1f}")

```

Run this three times. How much does the cluster mean vary? Why?

4. Fill in the table with your results.

Method	Run 1	Run 2	Run 3	Most stable?
SRS				
Stratified				
Cluster				
True mean	79.7			

5. Why does the stratified sample tend to produce a more stable estimate than the SRS? Why does the cluster sample vary the most?

3.4 Bias in Surveys

Even a good random sample can still give bad results if the survey is set up badly. Bias can happen at many steps, further down the line from when you pick people.

Sampling error is the normal difference between a sample and the population. Different samples can give different answers just by chance. This is not the same as bias. Sampling error gets smaller with a bigger sample.

The following types of bias are more serious because they do not shrink with larger samples. They push results systematically in a particular direction regardless of sample size.

Response bias occurs when respondents give answers that do not reflect their true beliefs or behaviors. This often happens when the topic is sensitive or when the presence of an interviewer creates social pressure. A survey on hand-washing habits conducted by a hygiene inspector at a food preparation facility will likely overstate how often workers wash their hands, because workers know what answer the inspector wants to hear.

Nonresponse bias occurs when individuals selected for the sample cannot be reached or choose not to participate, and those non-respondents differ systematically from those who do respond. A survey mailed to parents about school satisfaction will be completed at higher rates by engaged parents than by disengaged ones. The result overstates satisfaction among the parent community as a whole.

Undercoverage bias occurs when part of the population is excluded from the sampling frame entirely. A survey about internet access conducted through online banner advertisements cannot reach people who lack internet access. Those excluded are precisely the group most relevant to the question.

Wording effect bias occurs when the phrasing of a question influences the answer. Consider the difference between these two ways of asking essentially the same thing:

- "Do you support stricter emissions standards for vehicles?"
- "Do you support stricter emissions standards for vehicles, even if it means higher car prices and fewer model choices?"

The second version frames the question with costs attached, making a "no" response far more likely even among people who genuinely support the policy. Neutral, direct wording is essential for unbiased responses.

Exercise 5 Diagnosing Bias

Identify the most likely type of bias in each scenario and explain why it applies.

Working Space

1. A streaming platform sends a satisfaction survey to all subscribers but receives responses from only 8% of them. The platform reports that 91% of subscribers are satisfied.
2. A survey on recycling habits is conducted by going door-to-door in affluent neighborhoods only, because those streets are easier and safer to walk.
3. An interviewer asks employees: "Our new open-plan office is designed to encourage collaboration. How much has it improved your productivity?"
4. A survey on alcohol consumption is conducted at a community health fair by a team wearing hospital uniforms. Respondents significantly underreport their weekly intake.

Answer on Page 54

Data tables in SQL

Most organizations keep their data as tables inside a relational database management system (compared to pandas, CSV, or spreadsheets). Developers talk to those systems using a language called SQL (“Structured Query Language”).

Some relational database managers are pricey products you may have heard of before, such as Oracle or Microsoft SQL Server. Some are free, such as PostgreSQL or MySQL. These are server software that client programs talk to over a company’s network.¹

There is a library, called `sqlite`, that lets us create files that hold tables. We can use SQL to create, edit, and browse those tables. `sqlite` is free, fast, and very easy to install. We will use `sqlite` instead of a networked database management system.

If you look in your digital resources, you will find a file called `bikes.db`. We created this file using `sqlite`, and now you will use `sqlite` to access it.

In the terminal, get to the directory where `bikes.db` lives. To open the `sqlite` tool on that file:

```
1 > sqlite3 bikes.db
```

(If your system complains that there is no `sqlite3` tool, you need to install `sqlite`. See this website: <https://sqlite.org/>)

Please follow along: type each command shown here into the terminal and see what happens.

We mostly run SQL commands in this tool, but there are a few non-SQL commands that all start with a period. To see the tables and their columns, you can run `.schema`:

```
1 sqlite> .schema
2 CREATE TABLE bike (bike_id int PRIMARY KEY, brand text, size int,
3     purchase_price real, purchase_date date, status text);
```

¹It should be noted that many notetaking and file storage applications allow you to run SQL-like queries to search your files to meet certain criteria. For example, Obsidian, the Markdown notetaking app, has a plugin called Dataview, which allows you to run searches for notes matching certain *metadata* attribute criteria. See more about this here: <https://github.com/blacksmithgu/obsidian-dataview>

That is the SQL command that we used to create the bike table. You can see all the columns and their types.

You want to see all the rows of data in that table?

```
1  sqlite> select * from bike;
2  4997391|GT|57|269.61|2009-05-03|rented
3  5429447|Cannondale|50|215.91|2002-02-17|broken
4  5019171|Trek|58|251.17|1985-07-11|rented
5  3000288|Cannondale|57|211.08|1993-01-05|broken
6  880965|GT|52|281.75|1995-08-02|available
7  ...
```

You will see 1000 rows of data!

The SQL language is not case-sensitive, so you can also write it like this:

```
1  sqlite> SELECT * FROM BIKE;
```

Often, you will see SQL with just the SQL keywords in all caps:

```
1  sqlite> SELECT * FROM bike;
```

The semicolon is not part of SQL, but it tells sqlite that you are done writing a command and that it should be executed.

SQL lets you choose which columns you would like to see. The asterisk (FIXME) used above signifies all columns, and the `bike_id, brand` only gets the bike's id and brand from the dataframe:

```
1  sqlite> SELECT bike_id, brand FROM bike;
2  4997391|GT
3  5429447|Cannondale
4  5019171|Trek
5  3000288|Cannondale
6  ...
```

Using WHERE, SQL lets you use conditions to decide which rows you would like to see, and can be combined with the common operators AND, OR, and NOT:

```
1  sqlite> SELECT * FROM bike WHERE purchase_date > '2009-01-01' AND brand = 'GT';
2  4997391|GT|57|269.61|2009-05-03|rented
3  326774|GT|56|165.0|2009-06-27|available
```

```

4 | 264933|GT|52|302.43|2009-07-09|available
5 | 5931243|GT|55|173.56|2009-11-26|rented
6 | 4819848|GT|51|221.71|2009-12-11|rented
7 | 9347713|GT|52|232.32|2009-06-13|available
8 | 3019205|GT|58|262.94|2009-08-22|available

```

Using DISTINCT, SQL lets you get just one copy of each value:

```

1 | sqlite> SELECT DISTINCT status FROM bike;
2 | rented
3 | broken
4 | available
5 |
6 | Busted
7 | Flat tire
8 | good
9 | out
10 | Rented

```

You can also edit these rows. For example, if you wanted every status that is Busted to be changed to broken, you can use an UPDATE statement with a SET:

```

1 | sqlite> UPDATE bike SET status='broken' WHERE status='Busted';
2 | sqlite> SELECT DISTINCT status FROM bike;
3 | rented
4 | broken
5 | available
6 | Flat tire
7 | good
8 | out
9 | Rented

```

You can insert new rows:

```

1 | sqlite> INSERT INTO bike (bike_id, brand, size, purchase_price, purchase_date,
2 |   status)
3 | ...> VALUES (1, 'GT', 53, 123.45, '2020-11-13', 'available');
4 | sqlite> SELECT * FROM bike WHERE bike_id = 1;
5 | 1|GT|53|123.45|2020-11-13|available

```

Note that the bike_id here must be *unique*.

You can delete rows:

```
1 sqlite> DELETE FROM bike WHERE bike_id = 1;
2 sqlite> SELECT * FROM bike WHERE bike_id = 1;
```

To get out of sqlite, type `.exit`.

Exercise 6 SQL Query

Execute an SQL query that returns the `bike_id` (no other columns) of every Trek bike that cost more than \$300.

Working Space

Answer on Page 54

4.1 Using SQL from Python

The people behind sqlite created a library for Python that lets you execute SQL and fetch the results from inside a python program.

Let's create a simple program that fetches and displays the bike ID and purchase date of every Trek bike that cost more than \$300.

Create a file called `report.py`:

```
1 import sqlite3 as db
2
3 con = db.connect('bikes.db')
4 cur = con.cursor()
5
6 cur.execute("SELECT bike_id, purchase_date FROM bike WHERE purchase_price > 330
7             ↪ AND brand='Trek'")
8 rows = cur.fetchall()
9
10 today = datetime.date.today()
11 for row in rows:
12     print(f"Bike {row[0]}, purchased {row[1]}")
13
14 con.close()
```

When you execute it, you should see:

```
1 > python3 report.py
2 Bike 4128046, purchased 2007-08-06
3 Bike 7117808, purchased 1995-03-12
4 Bike 7176903, purchased 1986-07-03
5 Bike 827899, purchased 2009-03-14
6 Bike 363983, purchased 1970-08-16
```


Advanced SQL and Database Management

If you recall from our basic SQL chapters, we have the ability to create sets of rows and columns that represent data. We can represent a set of 1000 bikes, each with a respective price, status, brand, and id, in the form of rows (you may hear the term *tuples*) and columns (attributes). In this chapter, we will build out a larger database, with more than one table, but, here's the twist: we will find a way to link them together, so that data can be searched across tables.

5.1 Keys

In the experimentation from our first chapter, you may have seen the term *primary key*. An attribute (or set of attributes) that can uniquely identify rows within a table are called a primary key. That may have just sounded like a set of buzzwords, so let's take a step back:

Take for example the following table:

StudentID	Email	SSN	FirstName	LastName	EnrollmentDate
1001	alice@univ.edu	123-45-6789	Alice	Smith	2024-01-15
1002	bob@univ.edu	987-65-4321	Bob	Johnson	2024-01-15
1003	carol@univ.edu	555-55-5555	Carol	Williams	2024-02-01

Table 5.1: STUDENT table demonstrating key types.

We can identify columns or combinations of columns that could be used to uniquely identify each row. We can identify pretty much infinite *super keys*, which are any set of attributes that uniquely identifies a row. What sets these apart is that they may include extraneous attributes that are not required for uniqueness. For example, in the STUDENT table, both {StudentID} and {StudentID, Email} are super keys, but the latter includes an unnecessary attribute for uniqueness. Keys that contain more than one attribute are called *composite keys*, and are represented in sequence with parentheses such as (StudentID, Email).

A *candidate key* is a minimal super key, meaning it has no extraneous attributes. In our STUDENT table, {StudentID} and {SSN} are candidate keys because they uniquely iden-

tify each row and do not contain unnecessary attributes.

From the candidate keys, a *primary key* is selected. By convention, the primary key is underlined. The primary key is the main candidate key chosen to uniquely identify each row in the table. In our STUDENT table, {StudentID} could be chosen as the primary key. Then, {SSN} would be considered an *alternate key*, which is a candidate key not selected as the primary key. The reason we select {StudentID} over {SSN} is that {SSN} has real-world representation, which we call a *natural key*. We do not want to conflate real world attributes with database management tools such as primary keys.

A real world example: What if we were using an email as the primary key? Then, if the email schema changes due to the business being bought out or renamed (this happened to one of our authors!), we are stuck! A change like <firstinitial><lastname>@company.com to <firstname>.<lastname>@company2.com would break the database.

This is why we introduce *surrogate keys*. Surrogate keys are keys with no real world meaning, often stable identifiers that do not change. This includes auto-incrementing integers, Universally Unique Identifiers (*UUIDs*), or Hex Codes that do not have the opportunity to change. In short, we prefer surrogate keys for primary keys to ensure stability and avoid issues with real-world attribute changes.

In a few sections we will explore how foreign keys allow us to link tables together using these primary keys.

5.2 Normalization

With all of these key types, our tables can get pretty large. When this happens, we will want to normalize our database to ensure efficiency and reduce **redundancy**.

We have a process called *normalization*, which involves organizing, reordering, and restructuring the columns and tables of a database to minimize redundancy and dependency. This can help

- improve performance by reducing the amount of redundant data stored, as databases reach high volumes of data.
- prevent anomalies, which are inconsistencies or errors that can occur when redundant data is updated, inserted, or deleted.
- maintain data integrity

Ultimately, there is one primary goal of normalization:

The table can only express one version of truth.

5.2.1 Functional Dependencies

Normalization rests on a single formal idea: the *functional dependency*. We say an attribute (or set of attributes) Y is functionally dependent on attribute X , written $X \rightarrow Y$, if knowing the value of X tells us exactly one corresponding value of Y . In other words, X *determines* Y .

For example, in our STUDENT table from earlier, $\{\text{StudentID}\} \rightarrow \{\text{Email}\}$, since each StudentID corresponds to exactly one Email. This is exactly why $\{\text{StudentID}\}$ can serve as a key: a candidate key is a minimal set of attributes that functionally determines every other attribute in the table.

Two particular kinds of functional dependency will help us reason through the normal forms:

- A *partial dependency* occurs when a non-key attribute depends on only part of a composite primary key, rather than the whole key.
- A *transitive dependency* occurs when a non-key attribute depends on another non-key attribute, rather than depending directly on the primary key.

Removing partial dependencies brings a table to 2NF, and removing transitive dependencies brings a table to 3NF.

Let's go through an example. In this book, we will only focus on the first three normal forms: 1NF, 2NF, and 3NF. There are videos in your digital resources that go deeper into the higher normal forms.

Below is table representing an intentionally poorly designed database representing lab equipment at a university laboratory.

Table 5.2: Equipment Checkout Log

StudentID	EquipmentID	CheckoutDate	StudentName	StudentEmail	StudentYear	EquipmentName	EquipmentLocation	LocationManager
1001	E42, E55	2026-09-20	Alex	alex@univ.edu	Junior	3D Printer, Oscilloscope	Lab B, Lab A	Maria, David
1002	E42	2026-09-21	Jordan	jordan@univ.edu	Senior	3D Printer	Lab B	Maria

1

5.2.2 1NF

A database is in first normal form (1NF) if it meets the following criteria:

¹Notice how small the text had to get to render the entire table within the page width, this is also a clear indication that something is off!

- Each column contains atomic values (no repeating groups or arrays).
- Each row is unique (no duplicate rows).

Notice how the table violates the first normal form (1NF) because it contains repeating groups and multiple values in a single column. In 1NF, each column should contain atomic values, and each row should be unique. Invalid attribute values include:

- Multiple EquipmentIDs in a single cell ({E42, E55})
- Multiple EquipmentNames in a single cell ({3D Printer, Oscilloscope})
- Multiple EquipmentLocations in a single cell ({Lab B, Lab A})
- Multiple LocationManagers in a single cell ({Maria, David})

To remedy this, we make a second row for student Alex: Now our table conforms to the

Table 5.3: Equipment Checkout Log [1NF]

StudentID	EquipmentID	CheckoutDate	StudentName	StudentEmail	StudentYear	EquipmentName	EquipmentLocation	LocationManager
1001	E42	2026-09-20	Alex	alex@univ.edu	Junior	3D Printer	Lab B	Maria
1002	E42	2026-09-21	Jordan	jordan@univ.edu	Senior	3D Printer	Lab B	Maria
1001	E55	2026-09-22	Alex	alex@univ.edu	Junior	Oscilloscope	Lab A	David

first normal form (1NF), with each column containing atomic values and each row being unique.

5.2.3 2NF

1NF was quite easy, now we move on to the second normal form (2NF), which requires the following:

- The table is in 1NF.
- No partial dependencies (no non-key attribute should depend on a part of a composite primary key).
- All non-key attributes are fully functionally dependent on the primary key.

For demonstration purposes, lets add a fourth entry into our table:

Table 5.4: Equipment Checkout Log [Pre-2NF]

StudentID	EquipmentID	CheckoutDate	StudentName	StudentEmail	StudentYear	EquipmentName	EquipmentLocation	LocationManager
1001	E42	2026-09-20	Alex	alex@univ.edu	Junior	3D Printer	Lab B	Maria
1002	E42	2026-09-21	Jordan	jordan@univ.edu	Senior	3D Printer	Lab B	Maria
1001	E55	2026-09-22	Alex	alex@univ.edu	Junior	Oscilloscope	Lab A	David
1001	E74	2026-09-23	Alex	alex@univ.edu	Junior	CNC Machine	Lab C	Larry

We can establish that the primary key of this table is the combination of {StudentID, EquipmentID, CheckoutDate}, as each student can check out multiple pieces of equipment, and each piece of equipment can be checked out by multiple students.

However, we have lots of repeated data in here:

- Student information (name, email, year) is repeated for each piece of equipment they check out.
- Equipment information (name, location, location manager) is repeated for each student who checks it out.

This redundancy indicates that our table is not in 2NF, as non-key attributes are partially dependent on the composite primary key. To achieve 2NF, we need to separate the student and equipment information into their own tables, leaving only the checkout details in the original table.

The new schemas will look like this:

Student (StudentID, StudentName, StudentEmail, StudentYear)
 Equipment (EquipmentID, EquipmentName, EquipmentLocation, LocationManager)
 Checkout (StudentID, EquipmentID, CheckoutDate)

<u>StudentID</u>	StudentName	StudentEmail	StudentYear
1001	Alex	alex@univ.edu	Junior
1002	Jordan	jordan@univ.edu	Senior

Table 5.5: Student Table [2NF]

<u>EquipmentID</u>	EquipmentName	EquipmentLocation	LocationManager
E42	3D Printer	Lab B	Maria
E55	Oscilloscope	Lab A	David
E74	CNC Machine	Lab C	Larry

Table 5.6: Equipment Table [2NF]

<u>StudentID</u>	<u>EquipmentID</u>	<u>CheckoutDate</u>
1001	E42	2026-09-20
1002	E42	2026-09-21
1001	E55	2026-09-22
1001	E74	2026-09-23

Table 5.7: Checkout Table [2NF]

Notice how each table now has a single, clear primary key: {StudentID} for Student, {EquipmentID} for Equipment, and {StudentID, EquipmentID} for Checkout. Every non-key attribute is now fully dependent on its table's key, with no partial dependencies remaining. 2NF often splits up a table into multiple tables to remove partial dependencies and serves as a stepping stone to 3NF, which further eliminates transitive dependencies.

5.2.4 3NF

To transform a table from 2NF to 3NF, we need to meet the following conditions:

- The table must already be in 2NF (and, consequently, in 1NF).
- There should be no *transitive dependencies*, meaning non-key attributes should not depend on other non-key attributes.

Looking at our table, we see a transitive dependency:

$\text{EquipmentID} \rightarrow \text{EquipmentLocation} \rightarrow \text{LocationManager}$

These are all stored in the same table, which creates a transitive dependency that violates 3NF.

We now must make a new table to remove this transitive dependency, ensuring that each non-key attribute depends only on the primary key of its table.

Equipment (EquipmentID, EquipmentName, EquipmentLocation)
 Location (EquipmentLocation, LocationManager)

<u>EquipmentID</u>	EquipmentName	EquipmentLocation
E42	3D Printer	Lab B
E55	Oscilloscope	Lab A
E74	CNC Machine	Lab C

Table 5.8: Equipment Table [3NF]

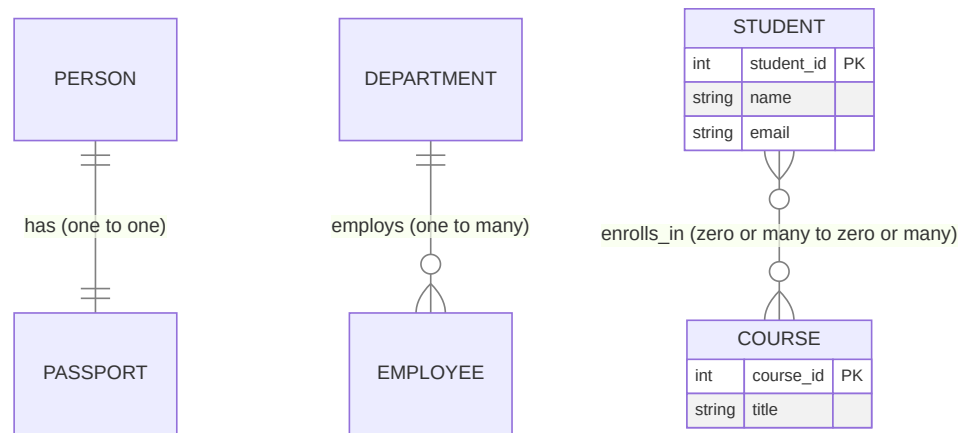
<u>EquipmentLocation</u>	LocationManager
Lab B	Maria
Lab A	David
Lab C	Larry

Table 5.9: Location Table [3NF]

FIXME exercise transform a full table up to 3NF

5.3 Entity-Relationship Diagram

FIXME Add an ER diagram for the above example with lab equip.



5.4 Relational Algebra Basics

5.5 Relating Two Tables

Now let's say we have two tables, **Equipment** and **Location**, and we want to relate them using a *foreign key*. The **Equipment** table has a column **EquipmentLocation** that references the primary key **EquipmentLocation** in the **Location** table. This allows us to link each piece of equipment to its corresponding location.

Let's first create the **Equipment** table with a foreign key referencing the **Location** table.

```

1 CREATE TABLE Location (
2     EquipmentLocation VARCHAR(50) PRIMARY KEY,
3     LocationManager VARCHAR(50)
4 );
5
6 CREATE TABLE Equipment (
7     EquipmentID VARCHAR(50) PRIMARY KEY,
8     EquipmentName VARCHAR(50),
9     EquipmentLocation VARCHAR(50),
10    FOREIGN KEY (EquipmentLocation) REFERENCES Location(EquipmentLocation)

```

11

);

We *constrain* the values in the `EquipmentLocation` column of the `Equipment` table to only those that exist in the `Location` table, ensuring referential integrity between the two tables.

Using relational algebra, we can say that

$$\pi_{\text{EquipmentLocation}}(\text{Equipment}) \subseteq \pi_{\text{EquipmentLocation}}(\text{Location})$$

All values in the `EquipmentLocation` column of the `Equipment` table must exist in the `EquipmentLocation` column of the `Location` table. However, it does not guarantee that every location in the `Location` table has corresponding equipment in the `Equipment` table. For example, here are the two tables again, with altered data

<u>EquipmentLocation</u>	LocationManager
Lab B	Maria
Lab A	David
Lab C	Larry

Table 5.10: Location Table

<u>EquipmentID</u>	EquipmentName	EquipmentLocation
E42	3D Printer	Lab B
E55	Oscilloscope	Lab A

Table 5.11: Equipment Table

Table constraints do not break when foreign key values are missing in the referencing table. For example, even though “Lab C” exists in the `Location` table, it does not appear in the `Equipment` table, and this is perfectly valid. However, if we tried to insert a row into the `Equipment` table with an `EquipmentLocation` value that does not exist in the `Location` table (such as “Lab D”), the database would reject it due to the foreign key constraint.

Diodes, Rectifiers, and Photovoltaics

Early in this course, we discussed some basic electronics components: resistors, capacitors, and inductors. In this chapter, we are going to introduce a semiconductor: *Diodes* ensure that current flows in only one direction.

You will learn a lot about semiconductors from studying diodes. We can then apply what you have learned to solar cells, transistors, and integrated circuits like CPUs.

We are going to focus on semiconductors made of silicon. You can make semiconductors out of other materials, but silicon has proved to be very versatile and cost-effective.

6.1 Silicon

A silicon atom has 14 protons and 14 electrons. The inner 10 electrons are arranged in two complete and stable shells. The outer four electrons are in an incomplete shell, which allows them to form bonds with other atoms. We call these *valence electrons* because they aren't tightly bound to their atoms. A complete outer shell would have eight electrons.

Atoms can share electrons to get complete shells. This is called *covalent bonding*. When atoms share electrons, they form a *covalent bond*. The electrons are shared between the atoms, and the atoms are held together by the shared electrons.

In a silicon crystal, each silicon atom shares its outer four electrons with four neighboring atoms. With its four electrons and an electron borrowed from each of its four neighbors, each silicon atom has eight electrons in its outer shell. This is a complete shell, and the atoms are stable. This is why glass is hard and stable. It also why electricity doesn't flow through glass – all the electrons are tightly bound to their atoms.

Phosphorus has five valence electrons. If you put a phosphorus atom into silicon crystal, it shares four of its electrons with its neighbors, but it has one electron left over after the complete shell of eight is built. As you add more phosphorus to a silicon crystal, it conducts more electricity. We call this *doping*.

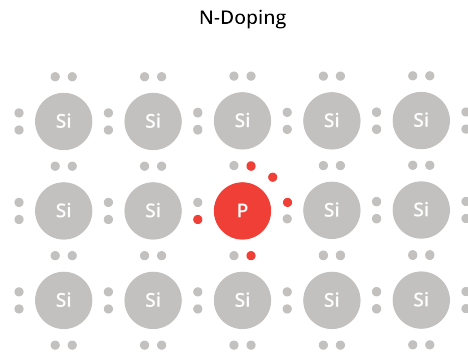


Figure 6.1: Phosphorus atom in a silicon crystal

Because this kind of doping frees up some electrons, which are negatively charged, we call this *n-doping*.

Boron has only three valence electrons. If you put a boron atom into silicon crystal, it shares three of its electrons with its neighbors, which leaves a "hole" that would like to be filled by an electron. We call this *p-doping*.

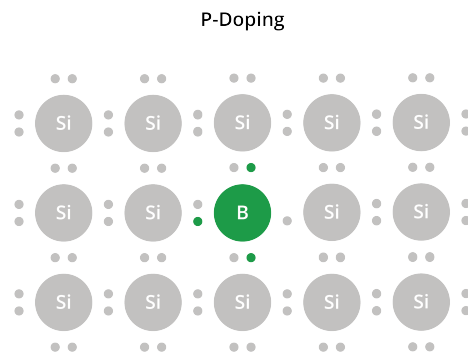


Figure 6.2: Boron atom in a silicon crystal

Both n-doped and p-doped silicon are conductors of electricity. In n-doped silicon, we say the loose electrons are the *carriers* of the current. In p-doped silicon, we say the holes are the carriers.

Note that doped silicon is still mostly silicon. In normal doping, maybe 1 in 10,000 atoms are replaced with boron or phosphorus.

6.2 Diodes

A diode is a semiconductor device that allows current to flow in one direction only.

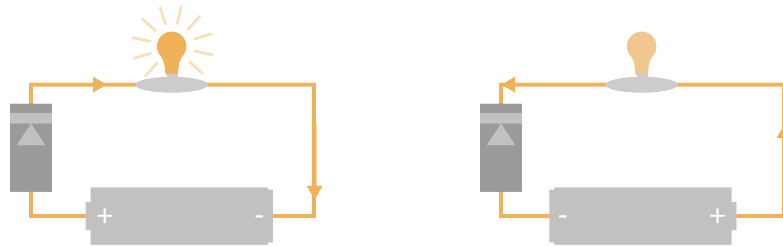


Figure 6.3: Current flows in one direction through the diode, not in the other direction.

Diodes can be made with silicon that has been n-doped and p-doped. A diode looks like this:



Figure 6.4: What Diodes Look Like

We say that current flows from the positive terminal to the negative terminal of a battery. However, remember that the electrons are actually moving from the negative terminal to the positive terminal. This is a common source of confusion in any discussion of diodes.

One end of the diode is called the *anode*. The anode is p-doped. The other end is called the *cathode*. The cathode is n-doped.

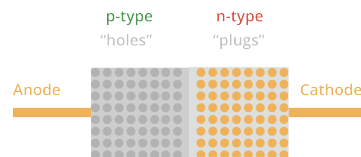


Figure 6.5: A p-n Diode

The region close to where the anode and cathode meet is called the *depletion region*. In this space, the spare electrons in the cathode (the n-doped material) migrate over to fill the holes in anode (the p-doped material). This creates an electric field: there are more electrons than protons on the anode side, and more protons than electrons on the cathode side. (How strong is this field? It is typically about 0.7 volts.)

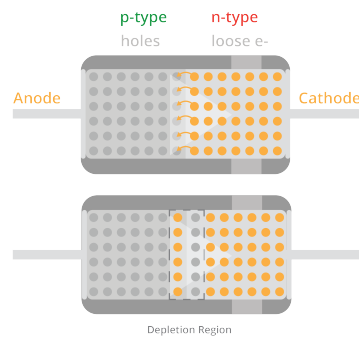


Figure 6.6: The depletion region in a diode

What happens if we overpower that 0.7 volts? What if we put 2 volts in the opposite direction? This shrinks the depletion region and the diode becomes a conductor. For example, if the negative end of a battery is connected to the cathode of a diode and the positive end is connected to the anode, current will flow through the diode.

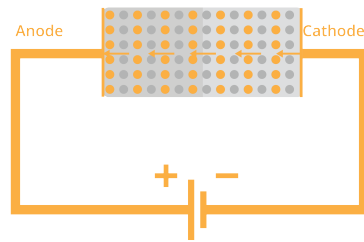


Figure 6.7: Electrons flow through the diode

If we reverse the battery, the depletion region expands, and the diode becomes an insulator.

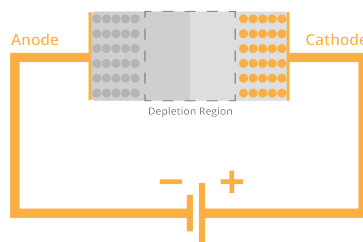


Figure 6.8: Reverse the electric field? No flow.

Here is the symbol for a diode:

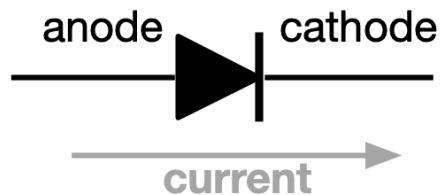


Figure 6.9: Diode Symbol

Current will flow in the direction of the arrow. It will not flow in the opposite direction.

6.2.1 Rectifiers

One of the really useful things we can do with a diode is to convert alternating current (AC) into direct current (DC). This is called rectification. Remember that AC pushes and pulls the electrons back and forth in the wire. Rectification ensures that the electrons only move in one direction.

A *half-wave rectifier* uses a single diode to ensure that the current flows in one direction through the load.



Figure 6.10: Half-wave Rectifier

Current flows in the direction of the arrow on the diode. If the voltage is reversed, the diode will block the current from moving.

A *full-wave rectifier* uses four diodes to swap the direction of the current through the load when necessary.

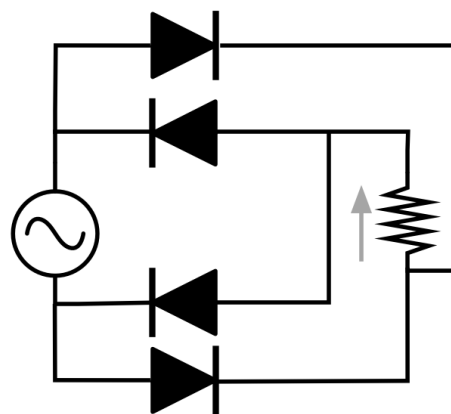


Figure 6.11: Full-wave Rectifier

6.2.2 Photovoltaics

Photovoltaics (or "Solar cells") are similar to diodes. They have a p-n junction with a 0.7 volt electrical field. The difference is that sunlight is allowed to shine on the cathode

side. When an atom is struck by a photon of light energy, the electron becomes excited. Sometimes it is excited enough to leap across that 0.7 volt barrier.

When it leaps across, there is an extra electron on the anode side of the barrier and an extra proton on the cathode side. The electron is pushed back to where it came from, but it can't go back over the barrier (unless more than 0.7 volts of charge have built up across the barrier). So, it goes the long way: through the load.

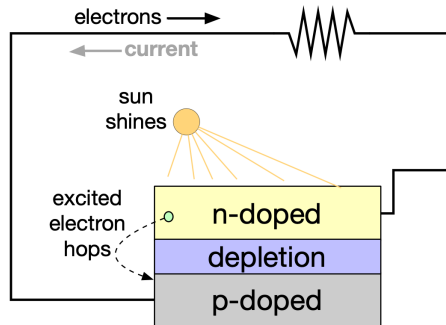


Figure 6.12: Solar Cell

Answers to Exercises

Answer to Exercise 1 (on page 20)

1. The population is all households in the city.
2. The frame is the city's water billing records.
3. The sample is the 200 households selected and interviewed.
4. No. A census would require interviewing every household in the city. Only 200 were selected here.

Answer to Exercise 2 (on page 21)

- **Library volunteers:** Census. With only 8 volunteers, asking all of them takes almost no effort.
- **Battery lifespan:** Sample. Testing can destroy the batteries, so a census would leave nothing to sell.
- **Commute times:** Sample. The population of 40 million is far too large for a full census to be practical.
- **Dietary restrictions:** Census. With only 22 guests, collecting complete information is straightforward and important for planning.

Answer to Exercise 3 (on page 23)

1. Number the 300 drivers from 001 to 300. Enter a random number table at any point and read three-digit numbers. Select any driver whose number falls between 001 and 300, skipping values from 301 to 999 and any numbers that have already been selected, until 25 drivers have been chosen. Alternatively, use `randInt(1,300,25)` on a TI-84, ignoring repeats.
2. This is a convenience sample. Drivers who are easy to reach at shift changes may work particular routes or shifts that differ systematically from others. For example,

they may have different safety exposure than drivers on overnight or rural routes.

3. No. An SRS requires that every possible group of 25 drivers from all 300 has an equal chance of being chosen. This procedure cannot produce a sample where all 25 come from the same depot, so not all groups of 25 are possible. This is a stratified sample, not an SRS.

Answer to Exercise 4 (on page 27)

- **Supermarket:** Cluster sampling. The stores are clusters, which means entire clusters are included once selected.
- **University:** Stratified random sampling. The four faculties are strata. An SRS is taken from each.
- **Water utility:** Systematic sampling. Random start followed by every 25th unit.
- **Film festival:** Simple random sample. Every possible group of 90 attendees is equally likely.

Answer to Exercise 5 (on page 30)

1. Nonresponse bias. The 92% who did not respond are unlikely to be satisfied in the same proportions as those who did. Highly satisfied and highly dissatisfied subscribers are both more motivated to respond than those with neutral feelings, but the pattern of non-response is unknown and the reported figure cannot be trusted.
2. Undercoverage bias. Residents of lower-income neighborhoods are excluded from the frame entirely. Recycling behavior may differ systematically by neighborhood type, so the results cannot generalize to the full population.
3. Wording effect bias. The question assumes the office has improved productivity and asks only "how much." Respondents are not given a neutral opportunity to say productivity has not changed or has declined.
4. Response bias. The formal appearance of the interviewers creates social pressure to give a health-conscious answer. Respondents adjust their reported behavior to match what they believe the interviewers expect.

Answer to Exercise 6 (on page 34)

```
1 SELECT bike_id FROM bike WHERE purchase_price > 330 AND brand='Trek'
```




INDEX

alternate key, 38
anode, 47

bias, 21
biased, 21
boron, 46

candidate key, 37
carriers, 46
cathode, 47
census, 19
cluster sampling, 25
Clusters, 26
clusters, 26
cognitive biases, 3
composite keys, 37
convenience sample, 22
convenience sampling, 26
covalent bond, 45
covalent bonding, 45

depletion region, 48
determines, 39
diode, 45
Diodes, 45
doping, 45

foreign key, 43
frame, 19
full-wave rectifier, 50
functional dependency, 39

Groupthink, 5

half-wave rectifier, 50
Haversine formula, 15
haversine formula, 15

In-group favoritism, 4

Judgmental sampling, 22
judgmental sampling, 22

latitude, 13
Longitude and Latitude
coordinates of, 14

n-doped, 46
n-doping, 46
natural key, 38
Nautical Mile, 15
nautical mile, 15
Nonresponse bias, 29
nonresponse bias, 29
normalization, 38

p-doped, 46
p-doping, 46
partial dependency, 39
phosphorus, 45
photovoltaics, 51
population, 19
primary key, 37, 38
quota sampling, 26

- rectifier, [50](#)
- Response bias, [29](#)
- response bias, [29](#)

- sample, [19](#)
- sample survey, [19](#)
- sampling
 - with replacement, [22](#)
 - without replacement, [22](#)
- Sampling error, [29](#)
- sampling error, [29](#)
- sampling with replacement, [22](#)
- sampling without replacement, [22](#)
- Self-serving bias, [4](#)
- silicon, [45](#)
- simple random sample, [22](#)
- Snowball sampling, [26](#)
- snowball sampling, [26](#)
- solar cells, [51](#)
- SQL, [31](#)
- SQLite, [31](#)
- Strata, [26](#)
- strata, [26](#)
- stratified random sampling, [24](#)
- super keys, [37](#)
- surrogate keys, [38](#)
- systematic sampling, [24](#)

- The bandwagon effect, [5](#)
- transitive dependencies, [42](#)
- transitive dependency, [39](#)

- undercoverage, [29](#)
- Undercoverage bias, [29](#)
- UUIDs, [38](#)

- valence electrons, [45](#)
- voluntary response sample, [22](#)

- Wording effect bias, [29](#)
- wording effect bias, [29](#)